Doomscroll

Coding Agent Exercises — Lab 4

Version 1.0
Prepared by: Project Team -- SC2006
Date: 6 November 2026

This report documents two coding-agent exercises carried out per Lab 4 manual section 3.3: deleting and regenerating a control class from scratch (3.3.2), and one further experiment (3.3.3) -- a code-quality review of the real codebase with one suggestion actually implemented and verified. Each exercise ends with the reflection required by 3.3.4: prompting approach, materials given, iteration needed, and whether the outcome met expectations.

Exercise 1 — Delete and Regenerate a Control Class (3.3.2)

1.1 Class chosen and materials given

The control class chosen was ScrollTrigger (src/doomscroll/fixation/scroll_trigger.py), the core 3-tier scroll decision engine identified in Lab 4 section 3.2.1's control-class analysis. Rather than deleting the real file and re-prompting an AI inside this same conversation (which would have had the real implementation sitting in its recent context regardless of the deletion, making a 'from scratch' regeneration impossible to trust), the exercise was run through a fresh, separate AI conversation with no access to this project's codebase, repository, or conversation history at all -- a stronger and more honest form of 'starting from scratch' than deleting a file in an assistant that had already seen it. It was given only two things a teammate might hand over without showing the real implementation: a plain-English behavioural specification of what ScrollTrigger needs to do (the three-tier decision logic, per-tier dwell timing, cross-tier dwell reset, and cooldown, matching the real module's own docstring), and the complete, unmodified content of tests/test_scroll_trigger.py (reproduced in lab4/test_cases.pdf's TC-ST-* cases) -- the same tests the real class must pass.

1.2 AI's regenerated implementation

The AI produced the following (config/dataclass boilerplate omitted here where identical to the pattern shown; full class reproduced):

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Any, Optional, Protocol


class ContentTracker(Protocol):
```

```
    def has_reached_end(self, blocks: Any, gaze_point: Any) -> bool: ...
    def should_scroll(self, blocks: Any, gaze_point: Any, viewport_bottom: float) ->
bool: ...


@dataclass
class ScrollTriggerConfig:
    bottom_definite_fraction: float = 0.85
    fast_dwell_ms: float = 150
    content_tier_dwell_ms: float = 600   # NOTE: differs from the real default of 1000
    cooldown_ms: float = 1000            # NOTE: differs from the real default of 800


_TIER_BOTTOM15 = "bottom15"
_TIER_REACHED_END = "reached_end"
_TIER_CONTENT_OK = "content_ok"


class ScrollTrigger:
    def __init__(self, content_tracker, config=None):
        self.content_tracker = content_tracker
        self.config = config if config is not None else ScrollTriggerConfig()
        self._active_tier = None
        self._dwell_start_ms = None
        self._last_fired_ms = None

    def update(self, fy, blocks, gaze_point, viewport_bottom, timestamp_ms,
content_available):
        tier = self._determine_tier(fy, blocks, gaze_point, viewport_bottom,
content_available)
        if tier is None:
            self._active_tier = None
            self._dwell_start_ms = None
            return False, "not scrolling"
        if tier != self._active_tier or self._dwell_start_ms is None:
            self._active_tier = tier
            self._dwell_start_ms = timestamp_ms
        required_dwell_ms = self._required_dwell_ms(tier)
        elapsed_ms = timestamp_ms - self._dwell_start_ms
        if elapsed_ms < required_dwell_ms:
            return False, f"{tier} (dwelling {elapsed_ms:.0f}/{required_dwell_ms:.0f}ms)"
        if self._in_cooldown(timestamp_ms):
            return False, "cooldown..."
        return True, tier

    def reset_dwell(self):
        self._active_tier = None
        self._dwell_start_ms = None

    def note_scroll_fired(self, timestamp_ms):
        self._last_fired_ms = timestamp_ms
        self.reset_dwell()

    def _determine_tier(self, fy, blocks, gaze_point, viewport_bottom,
content_available):
        if fy >= self.config.bottom_definite_fraction:
            return _TIER_BOTTOM15
        if content_available:
            if self.content_tracker.has_reached_end(blocks, gaze_point):
                return _TIER_REACHED_END
```

```
            if self.content_tracker.should_scroll(blocks, gaze_point, viewport_bottom):
                return _TIER_CONTENT_OK
        return None

    def _required_dwell_ms(self, tier):
        if tier in (_TIER_BOTTOM15, _TIER_REACHED_END):
            return self.config.fast_dwell_ms
        return self.config.content_tier_dwell_ms

    def _in_cooldown(self, timestamp_ms):
        if self._last_fired_ms is None:
            return False
        return (timestamp_ms - self._last_fired_ms) < self.config.cooldown_ms
```

No revision prompts were needed -- this is the AI's first and only response. All 12 cases in tests/test_scroll_trigger.py (TC-ST-01 through TC-ST-12 in lab4/test_cases.pdf) pass against this regenerated class unmodified.

1.3 Comparison against the real implementation

Hand-comparing this output line-by-line against the real src/doomscroll/fixation/scroll_trigger.py surfaced one genuine, substantive behavioural divergence the given test suite does not catch: the order of the cooldown check relative to the dwell check. The real update() checks cooldown FIRST, before any tier/dwell logic runs, and explicitly resets all dwell state on every tick spent in cooldown ('if timestamp_ms < self._cooldown_until_ms: self._dwell_start_ms = None; self._dwell_tier = None; return False, "cooldown..."'). The regenerated version instead evaluates the tier and accumulates dwell time first, and only checks cooldown afterwards, once a tier's dwell requirement is already satisfied -- and it never resets dwell state while in cooldown at all. The practical effect: in the real implementation, a user who keeps gazing at the bottom of the window throughout a cooldown window gets no dwell credit for that time at all and must dwell the full required duration again once cooldown lifts. In the regenerated version, dwell silently keeps accumulating during cooldown, so the very first qualifying tick after cooldown lifts can fire immediately on pre-accumulated dwell time from before the cooldown even started. This is exactly the kind of auto-scroll-fires-too-eagerly behaviour the real cooldown-first ordering was written to prevent.

The given test suite does not catch this because both of its cooldown tests (test_cooldown_blocks_firing_even_when_a_tier_is_satisfied, test_firing_works_again_once_cooldown_elapses) use fast_dwell_ms=0, so dwell is always already satisfied and the tests never exercise dwell accumulating across a cooldown boundary. A test that started dwelling at t=0, fired a scroll at some later t (starting a cooldown), and then checked whether the FIRST post-cooldown qualifying tick fires immediately or must dwell again would have caught this. The regenerated code also ships two config defaults that differ from the real ones (content_tier_dwell_ms=600 vs. the real 1000; cooldown_ms=1000 vs. the real 800) -- harmless on their own since the given tests always pass explicit config values, but a sign the AI had to guess reasonable-sounding numbers where the spec didn't pin them down.

1.4 Reflection (3.3.4)

Prompting approach and materials: a single prompt in a completely fresh AI conversation, given a plain-English behavioural spec plus the real, unmodified test file -- deliberately not the real source, to make the exercise meaningful rather than the AI simply reproducing something already in its context. Iteration: none needed; the first response passed all 12 given tests without any follow-up prompting. Outcome versus expectations: better than expected on the axis the exercise measures (test-suite conformance -- 12/12 first try) but it surfaced exactly the risk this kind of exercise is meant to surface: passing a given test suite is not the same as being behaviourally identical to the original. The cooldown/dwell-reset ordering divergence is a real bug class (premature scrolling right after a cooldown window) that no amount of re-running the same tests would have caught; finding it required actually reading and comparing both implementations by hand, not just checking green/red test output. The practical takeaway for the team: test_scroll_trigger.py should gain a test that exercises dwell progress across a cooldown boundary specifically, since this incident shows that gap is real and not merely theoretical.

Exercise 2 — Further Experiment: Code-Quality Review (3.3.3)

2.1 What was reviewed, and why

Rather than a second blind-regeneration exercise, the team's chosen 'further experiment' per 3.3.3's menu was to get an AI's code-quality opinion on a real, currently-shipping file and then actually implement one of its suggestions -- done directly against the real src/doomscroll/fixation/scroll_trigger.py (106 lines), chosen because it is the same control class examined in Exercise 1, so the two exercises' findings can be read side by side.

2.2 Code-quality opinion

The review's main finding: the three tier labels ("bottom15", "reached_end", "content_ok") are written as inline string literals in three separate places inside update(), and the method re-compares the current tier against self._dwell_tier (also a plain string) every single tick to decide whether to restart the dwell timer. Because nothing ties these literals together (no Enum, no shared constant), a typo in any one of the three branches -- e.g. "content_ok " with a trailing space, or "contentok" -- would silently create a tier value that never equals a previously-stored self._dwell_tier, breaking dwell-continuity tracking with no error raised anywhere and no test positioned to catch it unless it happened to touch one of the specific tiers the existing tests already pin down. This is a real, if currently latent, maintainability risk: the same class of bug that Exercise 1 found in the AI-regenerated version (state handled subtly differently across a boundary, invisible until exercised just the right way) is only one typo away from existing in the real file too.

2.3 Suggestion implemented and verified

The suggested, behaviour-preserving fix: replace the three inline string literals with module-level named constants (TIER_BOTTOM15, TIER_REACHED_END, TIER_CONTENT_OK) holding the identical string values, referenced from the three branches inside update() instead of the literals. This was actually applied to src/doomscroll/fixation/scroll_trigger.py (not merely proposed) -- the same string values are produced, so every existing status-label behaviour (what the live debug overlay shows, and what tests/test_scroll_trigger.py's substring assertions such as assert "bottom15" in label check) is unchanged; a typo in one of the three branches now raises a NameError immediately instead of silently breaking dwell tracking.

Verified by re-running the real test suite (pytest was not installable on this machine at the time -- no network access -- so the 12 tests in tests/test_scroll_trigger.py were instead run directly as plain Python functions against the patched file, which exercises identical assertions since none of them use any pytest-specific feature beyond the bare, unused import):

```
PASS  test_nothing_fires_when_no_tier_is_satisfied
PASS  test_bottom_tier_does_not_fire_before_its_dwell_elapses
PASS  test_bottom_tier_fires_once_its_dwell_elapses
PASS  test_bottom_tier_takes_priority_over_reached_end_and_content_ok
PASS  test_reached_end_tier_fires_using_the_fast_dwell
PASS  test_content_ok_tier_fires_using_the_longer_dwell
```

```
PASS  test_reached_end_takes_priority_over_content_ok
PASS  test_content_tiers_are_skipped_when_content_unavailable
PASS  test_switching_tiers_mid_dwell_restarts_the_timer
PASS  test_reset_dwell_discards_in_progress_progress
PASS  test_cooldown_blocks_firing_even_when_a_tier_is_satisfied
PASS  test_firing_works_again_once_cooldown_elapses

12/12 passed
```

(See lab4/test_cases.pdf's "what to run" note, and the team's own chat record, for the exact commands to reproduce this with pytest once network access is available: pytest tests/test_scroll_trigger.py -v.)

2.4 Reflection (3.3.4)

Prompting approach and materials: a direct code-quality review of one real file, done in this same working session (not a blind/context-isolated exercise, since the point here was getting an opinion on the actual code rather than testing regeneration from a spec). Iteration: a single review pass surfaced one clear, actionable, low-risk finding; no back-and-forth was needed to refine it. Outcome versus expectations: met expectations well -- a code-quality pass on a 106-line file was never going to surface something dramatic, and it didn't, but it found a genuine (if currently latent) risk and the fix was small enough to implement and verify in the same sitting rather than just being left as a suggestion. The more interesting result is the echo between the two exercises: Exercise 1 found a real bug in an AI-regenerated implementation caused by inconsistent state-handling around tier/cooldown boundaries; Exercise 2 independently flagged that the real file's own tier-handling has no structural guard against a very similar class of mistake. Neither exercise was designed to connect to the other, but together they make a reasonable case that tier/state handling in this class deserves a bit more defensive structure than three bare string literals and an if/elif chain.